



USB Drivers in Linux

Data Transfer to and from USB Devices

Part—13

This article, which is part of the series on Linux device drivers, continues from the previous two articles. It details the ultimate step of data transfer to and from a USB device, using your first USB driver in Linux.

Pugs continued, “To answer your question about how a driver selectively registers or skips a particular interface of a USB device, you need to understand the significance of the return value of the *probe()* callback.” Note that the USB core would invoke *probe* for all the interfaces of a detected device, except the ones which are already registered—thus, while doing it for the first time, it will probe for all interfaces. Now, if the probe returns 0, it means the driver has registered for that interface. Returning an error code indicates not registering for it. That’s all. “That was simple,” commented Shweta.

“Now, let’s talk about the ultimate—data transfers to and from a USB device,” continued Pugs.

“But before that, tell me, what is this `MODULE_DEVICE_TABLE`? This has been bothering me since you explained the USB device ID table macros,” asked Shweta, urging Pugs to slow down. “That’s trivial stuff. It is mainly for the user-space *depmod*,” he said. ‘Module’ is another term for a driver, which can be dynamically loaded/unloaded. The macro `MODULE_DEVICE_TABLE` generates two variables in a module’s read-only section, which is extracted by *depmod* and stored in global map files under `/lib/modules/<kernel_version>`. Two such files are *modules.usbmap* and *modules.pcimap*, for USB and PCI device drivers, respectively. This enables auto-loading of these drivers, as we saw the *usb-storage*